



Progetto:

UrbanMind

Gruppo:

G22

Titolo del documento:

Sviluppo

Document Info

Doc. Name	D2-UrbanMindSviluppo	Doc. Number	D2 V1.4
Description	Documento di sviluppo dell'applicazione		



Indice

UrbanMind	1
Indice	2
Scopo del documento	3
1. Web APIs	4
2. Implementation	17
2.1. Repository Organization	17
Struttura Directory Principale	17
Dettaglio Organizzazione Backend	18
Dettaglio Organizzazione Frontend	18
Documentazione e Deploy	19
File di Configurazione	20
2.2. Branching Strategy e Organizzazione del Lavoro	20
2.3. Processo di Sviluppo	21
2.4. Architettura dell'Applicazione	22
2.5. Gestione delle Dipendenze e Deploy	22
3 Frontend	24
Architettura e Struttura	24
Pagine Pubbliche (Autenticazione)	24
1. LoginView - Pagina di Accesso	24
2. RoleSelectView - Selezione Ruolo	25
Area Cittadino	26
3. CitizenDashboardView - Dashboard Cittadino	26
4. BudgetView - Visualizzazione Budget	27
5. ProposalsView - Gestione Proposte	28
6.NewProposalView - Creazione Nuova Proposta	28
7. NewReportView - Creazione Segnalazione	29
Area Operatore	30
8. OperatorDashboardView - Dashboard Operatore	30
9. OperatorUsersView - Gestione Utenti	31
10. OperatorReportsView - Gestione Segnalazioni	31
11. OperatorBudgetView - Gestione Budget	32
12. OperatorProposalsView - Gestione Proposte	32
Tecnologie e Caratteristiche Tecniche	33
4 Deployment	33



Scopo del documento

Il presente documento (D2) ha lo scopo di definire l'architettura tecnica, l'infrastruttura di deployment e le specifiche delle interfacce di comunicazione del progetto UrbanMind. L'obiettivo è fornire una panoramica dettagliata sulle scelte implementative, illustrando in primo luogo la configurazione degli ambienti e dei servizi tramite approcci di Infrastructure-as-Code (Docker e piattaforme cloud come Render). In secondo luogo, il documento definisce in modo rigoroso i contratti delle API RESTful secondo lo standard OpenAPI 3.0, documentando i percorsi, i metodi e i payload necessari per la corretta interazione tra i moduli di Frontend e il Backend.



1. Web APIs

Le API del progetto UrbanMind sono state progettate e documentate secondo le specifiche standard **OpenAPI 3.0**. Questo contratto di interfaccia definisce chiaramente come il Frontend (sviluppato in Vue.js/React) comunica con il Backend (Node.js).

Di seguito è riportata la specifica completa che copre l'autenticazione tramite identità digitale, le funzionalità per il cittadino (segnalazioni, proposte, bilancio) e l'area riservata all'amministrazione.

```
None
openapi: 3.0.0
info:
  version: '1.0'
  title: "UrbanMind OpenAPI 3.0"
  description: API for managing municipal budgets, proposals and reports in smart cities.
  license:
    name: MIT
  contact:
    name: UrbanMind Team
    email: admin@urbanmind.com
servers:
  - url: http://localhost:3000/api
    description: Development server
  - url: https://urbanmind-backend.onrender.com/api
    description: Production server

paths:
  # Authentication
  /users/login:
    post:
      summary: User login
      description: Authenticate user and return JWT token
      tags:
        - Authentication
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required:
                - email
                - password
              properties:
                email:
                  type: string
                  format: email
                  description: User email address
```




```
        password:
          type: string
          description: User password
responses:
  '200':
    description: Login successful
    content:
      application/json:
        schema:
          type: object
          properties:
            success:
              type: boolean
            data:
              type: object
              properties:
                user:
                  $ref: '#/components/schemas/User'
                token:
                  type: string
  '401':
    description: Invalid credentials

/users/register:
  post:
    summary: User registration
    description: Register a new user in the system
    tags:
      - Authentication
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - name
              - email
              - password
            properties:
              name:
                type: string
                description: User full name
              email:
                type: string
                format: email
                description: User email address
              password:
                type: string
                minLength: 6
                description: User password
              role:
```



```
        type: string
        enum: [cittadino, operatore]
        default: cittadino
        description: User role
    city:
        type: string
        description: User city
    age:
        type: integer
        minimum: 16
        description: User age
responses:
  '201':
    description: User created successfully
    content:
      application/json:
        schema:
          type: object
          properties:
            success:
              type: boolean
            data:
              type: object
              properties:
                user:
                  $ref: '#/components/schemas/User'
                token:
                  type: string

# Budgets
/budgets:
  get:
    summary: Get all budgets
    description: Retrieve budgets with optional filters and pagination
    tags:
      - Budgets
    parameters:
      - in: query
        name: category
        schema:
          type: string
          enum: [infrastrutture, trasporti, ambiente, sociale, cultura, sicurezza,
istruzione, sanita, altro]
        description: Filter by budget category
      - in: query
        name: year
        schema:
          type: integer
          minimum: 2020
          maximum: 2050
        description: Filter by year
      - in: query
```



```
    name: status
    schema:
      type: string
      enum: [proposto, in_revisione, approvato, respinto, in_corso, completato]
    description: Filter by budget status
  - in: query
    name: page
    schema:
      type: integer
      minimum: 1
      default: 1
    description: Page number for pagination
  - in: query
    name: limit
    schema:
      type: integer
      minimum: 1
      maximum: 100
      default: 10
    description: Number of items per page
responses:
  '200':
    description: List of budgets
    content:
      application/json:
        schema:
          type: object
          properties:
            success:
              type: boolean
            data:
              type: object
              properties:
                budgets:
                  type: array
                  items:
                    $ref: '#/components/schemas/Budget'
            pagination:
              $ref: '#/components/schemas/Pagination'

post:
  summary: Create new budget
  description: Create a new municipal budget
  tags:
    - Budgets
  security:
    - bearerAuth: []
  requestBody:
    required: true
    content:
      application/json:
        schema:
```



```
      $ref: '#/components/schemas/BudgetInput'
responses:
  '201':
    description: Budget created successfully
    content:
      application/json:
        schema:
          type: object
          properties:
            success:
              type: boolean
            data:
              $ref: '#/components/schemas/Budget'

/budgets/{id}:
  get:
    summary: Get budget by ID
    description: Retrieve a specific budget by its ID
    tags:
      - Budgets
    parameters:
      - in: path
        name: id
        required: true
        schema:
          type: string
        description: Budget ID
    responses:
      '200':
        description: Budget details
        content:
          application/json:
            schema:
              type: object
              properties:
                success:
                  type: boolean
                data:
                  $ref: '#/components/schemas/Budget'
      '404':
        description: Budget not found

/budgets/stats/{year}:
  get:
    summary: Get budget statistics by year
    description: Retrieve budget statistics aggregated by category for a specific
year
    tags:
      - Budgets
    parameters:
      - in: path
        name: year
```



```
      required: true
      schema:
        type: integer
      description: Year for statistics
    responses:
      '200':
        description: Budget statistics
        content:
          application/json:
            schema:
              type: object
              properties:
                success:
                  type: boolean
                data:
                  $ref: '#/components/schemas/BudgetStats'

/budgets/compare/years:
  get:
    summary: Compare budgets across years
    description: Compare budget data between multiple years
    tags:
      - Budgets
    parameters:
      - in: query
        name: years
        required: true
        schema:
          type: string
        description: Comma-separated list of years to compare (e.g.,
"2023,2024,2025")
    responses:
      '200':
        description: Year comparison data
        content:
          application/json:
            schema:
              type: object
              properties:
                success:
                  type: boolean
                data:
                  type: object
                  properties:
                    comparison:
                      type: array
                      items:
                        $ref: '#/components/schemas/YearComparison'

/budgets/categories/list:
  get:
    summary: Get budget categories
```



```
description: Retrieve list of available budget categories with statistics
tags:
  - Budgets
responses:
  '200':
    description: Budget categories list
    content:
      application/json:
        schema:
          type: object
          properties:
            success:
              type: boolean
            data:
              type: object
              properties:
                categories:
                  type: array
                  items:
                    type: string
                details:
                  type: array
                  items:
                    $ref: '#/components/schemas/CategoryDetails'

/budgets/report/{format}:
get:
  summary: Generate budget report
  description: Generate budget report in CSV or JSON format
  tags:
    - Budgets
  parameters:
    - in: path
      name: format
      required: true
      schema:
        type: string
        enum: [csv, json]
      description: Report format (csv or json)
    - in: query
      name: year
      schema:
        type: integer
      description: Filter by year
    - in: query
      name: category
      schema:
        type: string
      description: Filter by category
  responses:
    '200':
      description: Report generated successfully
```



```
    content:
      text/csv:
        schema:
          type: string
          description: CSV report file
      application/json:
        schema:
          type: object
          properties:
            success:
              type: boolean
            data:
              type: object
              properties:
                budgets:
                  type: array
                  items:
                    $ref: '#/components/schemas/Budget'
                statistics:
                  $ref: '#/components/schemas/BudgetStats'

components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

  schemas:
    User:
      type: object
      required:
        - _id
        - name
        - email
        - role
      properties:
        _id:
          type: string
          description: User ID
        name:
          type: string
          description: User full name
        email:
          type: string
          format: email
          description: User email address
        role:
          type: string
          enum: [cittadino, operatore]
          description: User role
        city:
```



```
    type: string
    description: User city
  age:
    type: integer
    description: User age
  createdAt:
    type: string
    format: date-time
    description: Account creation date

Budget:
  type: object
  required:
    - _id
    - title
    - totalAmount
    - category
    - year
    - status
  properties:
    _id:
      type: string
      description: Budget ID
    title:
      type: string
      description: Budget title
    description:
      type: string
      description: Budget description
    totalAmount:
      type: number
      format: float
      description: Total budget amount in euros
    allocatedAmount:
      type: number
      format: float
      description: Already allocated amount in euros
    remainingAmount:
      type: number
      format: float
      description: Remaining available amount in euros
    category:
      type: string
      enum: [infrastrutture, trasporti, ambiente, sociale, cultura, sicurezza,
istruzione, sanita, altro]
      description: Budget category (mission)
    year:
      type: integer
      description: Budget year
    status:
      type: string
      enum: [proposto, in_revisione, approvato, respinto, in_corso, completato]
```




```
    description: Budget status
  department:
    type: string
    description: Responsible department
  manager:
    $ref: '#/components/schemas/User'
  startDate:
    type: string
    format: date
    description: Budget start date
  endDate:
    type: string
    format: date
    description: Budget end date
  createdAt:
    type: string
    format: date-time
    description: Creation timestamp
  updatedAt:
    type: string
    format: date-time
    description: Last update timestamp

BudgetInput:
  type: object
  required:
    - title
    - description
    - totalAmount
    - category
    - year
    - department
    - manager
    - startDate
    - endDate
  properties:
    title:
      type: string
      minLength: 5
      maxLength: 200
      description: Budget title
    description:
      type: string
      minLength: 10
      maxLength: 2000
      description: Budget description
    totalAmount:
      type: number
      format: float
      minimum: 0
      description: Total budget amount in euros
    category:
```



```
    type: string
    enum: [infrastrutture, trasporti, ambiente, sociale, cultura, sicurezza,
istruzione, sanita, altro]
    description: Budget category
  year:
    type: integer
    minimum: 2020
    maximum: 2050
    description: Budget year
  department:
    type: string
    description: Responsible department
  manager:
    type: string
    description: Manager user ID
  startDate:
    type: string
    format: date
    description: Budget start date
  endDate:
    type: string
    format: date
    description: Budget end date

BudgetStats:
  type: object
  properties:
    byCategory:
      type: array
      items:
        type: object
        properties:
          _id:
            type: string
            description: Category name
          totalBudget:
            type: number
            description: Total budget for category
          allocatedBudget:
            type: number
            description: Allocated budget for category
          remainingBudget:
            type: number
            description: Remaining budget for category
          count:
            type: integer
            description: Number of budgets in category
    total:
      type: object
      properties:
        totalBudget:
          type: number
```



```
        allocatedBudget:
          type: number
        remainingBudget:
          type: number
        count:
          type: integer

YearComparison:
  type: object
  properties:
    year:
      type: integer
      description: Year being compared
    byCategory:
      type: array
      items:
        type: object
        properties:
          _id:
            type: string
          totalBudget:
            type: number
          allocatedBudget:
            type: number
          remainingBudget:
            type: number
          count:
            type: integer
    total:
      type: object
      properties:
        totalBudget:
          type: number
        allocatedBudget:
          type: number
        remainingBudget:
          type: number
        count:
          type: integer

CategoryDetails:
  type: object
  properties:
    category:
      type: string
      description: Category name
    count:
      type: integer
      description: Number of budgets in category
    totalBudget:
      type: number
      description: Total budget for category
```



```
years:
  type: array
  items:
    type: integer
  description: Years with data for this category

Pagination:
  type: object
  properties:
    current:
      type: integer
      description: Current page number
    pages:
      type: integer
      description: Total number of pages
    total:
      type: integer
      description: Total number of items
    hasNext:
      type: boolean
      description: Whether there is a next page
    hasPrev:
      type: boolean
      description: Whether there is a previous page

Error:
  type: object
  properties:
    success:
      type: boolean
      example: false
    message:
      type: string
      description: Error message
    errors:
      type: array
      items:
        type: string
      description: Detailed error information
```



2. Implementation

L'applicazione è stata sviluppata utilizzando le seguenti tecnologie: **NodeJS** e **VueJS** per lo sviluppo full-stack, **MongoDB** per la gestione dei dati, con **Vite** come build tool per il frontend. Lo stack tecnologico è stato motivato dalle competenze pregresse del team e dall'adeguatezza di queste tecnologie per lo sviluppo di applicazioni web moderne con interfacce reattive e gestione efficiente dei dati. L'utilizzo di **Docker** attraverso docker-compose facilita il deployment e la gestione dell'ambiente di sviluppo.

2.1. Repository Organization

Il codice del progetto (<https://github.com/UrbanMind-ING-SW/UrbanMind>) è organizzato secondo la seguente struttura:

Struttura Directory Principale

Code

```
None
UrbanMind/
├── .DS_Store          # file di sistema macOS (da ignorare)
├── README.md          # documentazione principale del progetto
├── docker-compose.yml # configurazione containerizzazione Docker
├── /backend/          # cartella backend Node.js/Express.js
│   ├── .DS_Store      # file di sistema (da ignorare)
│   ├── package.json   # dipendenze e configurazione npm backend
│   ├── server.js       # entry point principale del server
│   └── /src/           # file sorgenti backend
│       └── app.js      # applicazione principale Express.js
├── /frontend/         # cartella frontend Vue.js
│   ├── index.html     # file HTML principale SPA
│   └── vite.config.js  # configurazione Vite per build e dev server
└── /docs/             # cartella documentazione del progetto
```



Dettaglio Organizzazione Backend

Basandosi sui feature branch sviluppati, la struttura backend prevista include:
Code

```
None
backend/
├─ package.json           # dipendenze Node.js, Express, MongoDB
├─ server.js              # entry point principale server
├─ /scripts/
│   └─ seedHistoricalData.js # popolazione dati esempio
└─ /src/
    ├─ app.js             # configurazione Express app
    ├─ /middleware/
    │   └─ auth.js        # autenticazione JWT middleware
    ├─ /models/           # schemi dati MongoDB/Mongoose
    │   ├─ User.js        # model utenti (citizen/operator)
    │   ├─ Budget.js      # model budget comunali
    │   ├─ Proposal.js    # model proposte cittadine
    │   └─ Report.js      # model segnalazioni urbane
    ├─ /routes/           # endpoint REST API
    │   ├─ users.js       # autenticazione, CRUD utenti
    │   ├─ budgets.js     # CRUD budget, statistiche, CSV
    │   ├─ proposals.js   # CRUD proposte, voti, commenti
    │   └─ reports.js     # CRUD segnalazioni, geolocalizzazione
    └─ /utils/            # utilità condivise
        ├─ db.js          # connessione MongoDB Atlas
        ├─ response.js    # formattori risposte API
        └─ validators.js  # validatori input dati
```

Dettaglio Organizzazione Frontend

Basandosi sui feature branch, la struttura frontend Vue.js prevede:
Code

```
None
frontend/
├─ package.json           # dipendenze Vue 3, TypeScript, Vite
├─ index.html             # template SPA principale
├─ vite.config.ts         # configurazione build/dev Vite
├─ tsconfig.json          # configurazione TypeScript
├─ eslint.config.ts       # linting rules
├─ ARCHITECTURE.md        # architettura frontend
├─ DESIGN_SYSTEM.md      # sistema design UI
├─ /public/              # assets statici
└─ /src/
    ├─ App.vue            # componente radice applicazione
    ├─ main.ts            # entry point TypeScript
    ├─ /assets/
    │   └─ main.css       # stili globali CSS
    └─ /components/      # componenti riusabili
```



```
|   |─ MainNavbar.vue      # barra navigazione principale
|   |─ Sidebar.vue        # menu laterale navigazione
|   |─ LeafletMap.vue     # mappa interattiva OpenStreetMap
|   |─ AddressAutocomplete.vue # autocomplete indirizzi
|   |─ /features/         # moduli funzionalità per ruolo
|   |─ /auth/views/       # autenticazione login/selezione ruolo
|   |─ /citizen/views/    # dashboard cittadino, proposte, budget
|   |─ /operator/views/   # dashboard operatore, gestione admin
|   |─ /router/           # routing SPA Vue Router
|   |─ index.ts           # configurazione router principale
|   |─ /routes/           # definizioni route per ruolo
|   |─ /stores/           # state management Pinia
|   |─ user.ts            # stato autenticazione utente
|   |─ proposals.ts       # stato proposte cittadine
|   |─ reports.ts         # stato segnalazioni urbane
|   |─ /services/
|   |   └─ api.ts         # client HTTP per API backend
|   |─ /composables/     # logica riusabile Vue 3
|   |   └─ useFetch.ts    # composabile chiamate API
|   |   └─ useForm.ts     # composabile gestione form
|   |   └─ useProposalFilters.ts # composabile filtri proposte
|   |─ /schemas/
|   |   └─ forms.ts       # validazione form TypeScript
```

Documentazione e Deploy

Code

```
None
docs/
|─ API_DOCUMENTATION.md    # guida API con esempi curl
|─ openapi.yaml            # specifica OpenAPI 3.0 completa
|─ ARCHITECTURE.md         # architettura sistema generale
|─ DATABASE_SETUP.md       # configurazione MongoDB Atlas
|─ DEVELOPMENT.md          # setup ambiente sviluppo
|─ DEPLOYMENT.md           # istruzioni deploy produzione

deploy/
|─ render.yaml             # Blueprint deploy Render automatico
|─ docker-compose.yml      # containerizzazione locale
|─ /backend/
|   └─ Dockerfile          # immagine container backend
|─ /frontend/
|   └─ Dockerfile          # immagine container frontend
```



File di Configurazione

Package Management:

- `backend/package.json`: Dipendenze Node.js, Express.js, MongoDB drivers, middleware
- `frontend/package.json`: Dipendenze Vue.js 3, Vue Router, Vite, librerie UI

Build e Deploy:

- `frontend/vite.config.js`: Configurazione build, proxy dev server, ottimizzazioni
- `docker-compose.yml`: Orchestrazione multi-container per ambiente completo
- `openapi.yaml`: Specifica OpenAPI 3.0 per documentazione API
- `render.yaml`: Blueprint deploy automatico su Render.com

Note sulla Struttura Attuale:

1. **Master Branch**: Contiene solo la struttura base di setup (file principalmente vuoti)
2. **Feature Branches**: Contengono l'implementazione reale delle funzionalità
3. **Separazione Netta**: Backend e frontend completamente separati per deployment indipendente
4. **Containerization**: Docker Compose per gestione ambiente sviluppo/produzione
5. **Modern Stack**: Utilizzo di Vite per build performanti e hot reload

Questa organizzazione supporta lo sviluppo modulare e scalabile, con chiara separazione delle responsabilità e facilità di deployment in diversi ambienti.

2.2. Branching Strategy e Organizzazione del Lavoro

Lo sviluppo ha visto una collaborazione attiva tra i membri del gruppo. Il lavoro è stato distribuito seguendo una logica modulare basata sui ruoli utente (Operatore/Utente) e sulle funzionalità specifiche.

Statistiche di sviluppo aggiornate:

- **Commits totali**:
 - 2 nel branch master
 - 62 nel dev
 - 116 in tutti gli altri sottobranch
- **Issues totali**: 26 (25 completate, 1 in corso)
- **Pull Requests**: 11 (tutte con merge andato a buon fine)
- **Branch attivi**: 15 feature branch paralleli
- **Periodo di sviluppo**: Dicembre 2025 - Febbraio 2026



Commit History:

- **Commit iniziale** (66deb5e): "Initial commit" - Setup repository
- **Setup struttura** (b1351aa): "Feat: initial arrangement of the project and the folders we will need"
- **Ultimo commit prima della consegna**(8093733): "Add comprehensive API documentation with OpenAPI 3.0 spec"

A livello di repository abbiamo optato per una strategia di branching **Feature Branch Workflow** con branch di integrazione. Durante lo sviluppo di nuove funzionalità abbiamo lavorato sui seguenti branch, derivandoli dal branch **master** e utilizzando **dev** come branch di integrazione:

Branch Infrastrutturali:

- **Feat/API**: sviluppo delle API backend e integrazione con MongoDB
- **Feat/login**: implementazione del sistema di autenticazione e autorizzazione

Branch Funzionalità Operatore:

- **Feat/operator-dashboard-layout**: sviluppo del layout principale della dashboard operatore
- **Feat/operator-homepage**: implementazione della homepage dedicata agli operatori
- **Feat/operator-budgets**: gestione dei budget e allocazione risorse per operatori
- **Feat/operator-proposals**: sistema di gestione proposte per operatori
- **Feat/operator-reports**: modulo di reporting e analytics per operatori
- **Feat/operator-users**: gestione utenti e permessi da parte degli operatori

Branch Funzionalità Utente:

- **Feat/user-dashboard-layout**: layout della dashboard per utenti finali
- **Feat/user-homepage**: homepage personalizzata per gli utenti
- **Feat/user-budget**: gestione budget personali degli utenti
- **Feat/user-proposals**: interfaccia per la creazione e gestione proposte utente
- **Feat/user-reports**: visualizzazione report e statistiche per utenti

Branch di Sviluppo e Integrazione:

- **dev**: branch di sviluppo per l'integrazione delle feature prima del deploy
- **master**: branch per le versioni stabili e funzionanti

2.3. Processo di Sviluppo

Il team ha adottato un approccio sistematico per lo sviluppo:

1. **Project Setup**: Creazione della struttura base del progetto con separazione frontend/backend
2. **Issue Tracking**: Ogni funzionalità è stata tracciata tramite GitHub Issues



3. **Feature Development:** Sviluppo isolato in branch dedicati
4. **Integration Testing:** Test di integrazione nel branch **dev**
5. **Pull Request Review:** Ogni feature è stata integrata tramite PR con review
6. **Continuous Integration:** Merge frequenti per evitare conflitti

2.4. Architettura dell'Applicazione

L'applicazione segue un'architettura disaccoppiata (Decoupled) con una netta separazione tra Frontend e Backend:

Frontend (Vue.js 3 + Vite):

- **Single Page Application (SPA):** Navigazione fluida gestita tramite Vue Router.
- **State Management:** Utilizzo di Pinia per la gestione centralizzata dello stato (autenticazione e dati globali).
- **Interfacce dedicate:** Dashboard differenziate per operatori e cittadini.
- **Data Visualization:** Integrazione con Chart.js per la rappresentazione grafica dei bilanci.
- **Build Tool:** Ottimizzazione tramite Vite per performance elevate sia in sviluppo che in produzione.

Backend (Node.js + Express.js):

- **API RESTful:** Endpoint standardizzati per la manipolazione delle risorse (CRUD).
- **Persistence Layer:** Utilizzo di MongoDB con interfaccia Mongoose per la modellazione dei dati.
- **Security:** Middleware di autenticazione basato su JWT.
- **Modularità:** Separazione della logica di business tra controller, rotte e modelli.
- **Environment:** Entry point configurabile per il deployment scalabile (es. Render).

2.5. Gestione delle Dipendenze e Deploy

Il progetto utilizza:

- **npm** per la gestione delle dipendenze separate per frontend e backend
- **Docker Compose** per containerizzazione e gestione ambiente
- **Vite** come build tool moderno per il frontend con hot reload
- **Render.com** come piattaforma di deploy cloud con Blueprint automatico
- **render.yaml** per configurazione deployment multi-servizio
- Configurazione modulare che supporta ambienti di sviluppo e produzione

Setup Attuale: Il progetto è in produzione e completamente funzionale, con:

- **Frontend deployed:** <https://urbanmind-frontend.onrender.com>
- **Backend deployed:** <https://urbanmind-backend.onrender.com/api>
- Database **MongoDB** Atlas configurato e popolato
- Deploy automatico da GitHub via **Render Blueprint**
- **15+** feature branch completamente sviluppati e integrati:
 - Sistema di autenticazione completo
 - Dashboard cittadino e operatore funzionali
 - CRUD completi per Budget, Proposals, Reports
 - API REST documentate con OpenAPI 3.0
- Branch **DEV** principale per development attivo
- Branch **MASTER** stabile per production releases



L'architettura microservizi è operativa con integrazione continua, documentazione API completa, e tutte le funzionalità core implementate e testate.



3 Frontend

L'applicazione frontend di UrbanMind è sviluppata utilizzando **Vue.js 3** con **TypeScript**, e si avvale di tecnologie moderne come **Vite** per il bundling, **Vue Router** per la navigazione e **Pinia** per la gestione dello stato. L'interfaccia è progettata per offrire un'esperienza utente fluida e responsive, con particolare attenzione all'accessibilità e all'usabilità su dispositivi mobili.

Architettura e Struttura

Il frontend segue un'architettura modulare organizzata per feature, con una chiara separazione tra le funzionalità pubbliche, quelle dedicate ai cittadini e quelle riservate agli operatori comunali. Il routing è gestito attraverso tre gruppi principali di route: **publicRoutes**, **citizenRoutes** e **operatorRoutes**.

Pagine Pubbliche (Autenticazione)

1. LoginView - Pagina di Accesso



Figura 1: schermata di login

La pagina di login rappresenta il punto di ingresso principale dell'applicazione. L'interfaccia presenta un design moderno e accattivante, con uno sfondo fotografico della città di Trento che crea un immediato senso di appartenenza territoriale. Il layout è caratterizzato da un pannello centrale semi-trasparente con effetto glassmorphism (backdrop blur), che garantisce una buona leggibilità mantenendo un aspetto visivamente gradevole.

La pagina offre tre modalità di accesso:



- **Autenticazione SPID:** permette l'accesso tramite il Sistema Pubblico di Identità Digitale, rispettando gli standard nazionali per l'identificazione digitale dei cittadini
- **Autenticazione CIE:** consente l'accesso tramite Carta d'Identità Elettronica, offrendo un'alternativa sicura e certificata
- **Accesso Test/Demo:** modalità dedicata per scopi dimostrativi e di testing, che reindirizza l'utente alla selezione del ruolo

L'interfaccia è completamente responsive, adattandosi perfettamente a schermi di diverse dimensioni, con transizioni fluide e feedback visivi all'interazione dell'utente.

2. RoleSelectView - Selezione Ruolo



Figura 2: schermata di selezione profilo (post-login)

Questa pagina intermedia permette all'utente di specificare con quale ruolo desidera accedere al sistema. Il design mantiene la coerenza visiva con la pagina di login, utilizzando lo stesso sfondo e stile di pannello.

L'utente può scegliere tra due ruoli:

- **Cittadino:** accesso all'area dedicata ai cittadini per consultare budget, creare proposte e segnalazioni
- **Operatore Comunale:** accesso all'area amministrativa per la gestione degli utenti, delle proposte e dei report

La pagina implementa una logica intelligente di autenticazione: tenta prima il login con credenziali predefinite per il ruolo selezionato e, se l'utente non esiste, procede automaticamente con la registrazione. Questa funzionalità è particolarmente utile in fase di demo e sviluppo. Durante il processo di autenticazione, vengono forniti feedback visivi all'utente (stati di caricamento, messaggi di errore) per garantire un'esperienza trasparente.



Area Cittadino

L'area cittadino è composta da cinque pagine principali, ciascuna dedicata a specifiche funzionalità:

3. CitizenDashboardView - Dashboard Cittadino

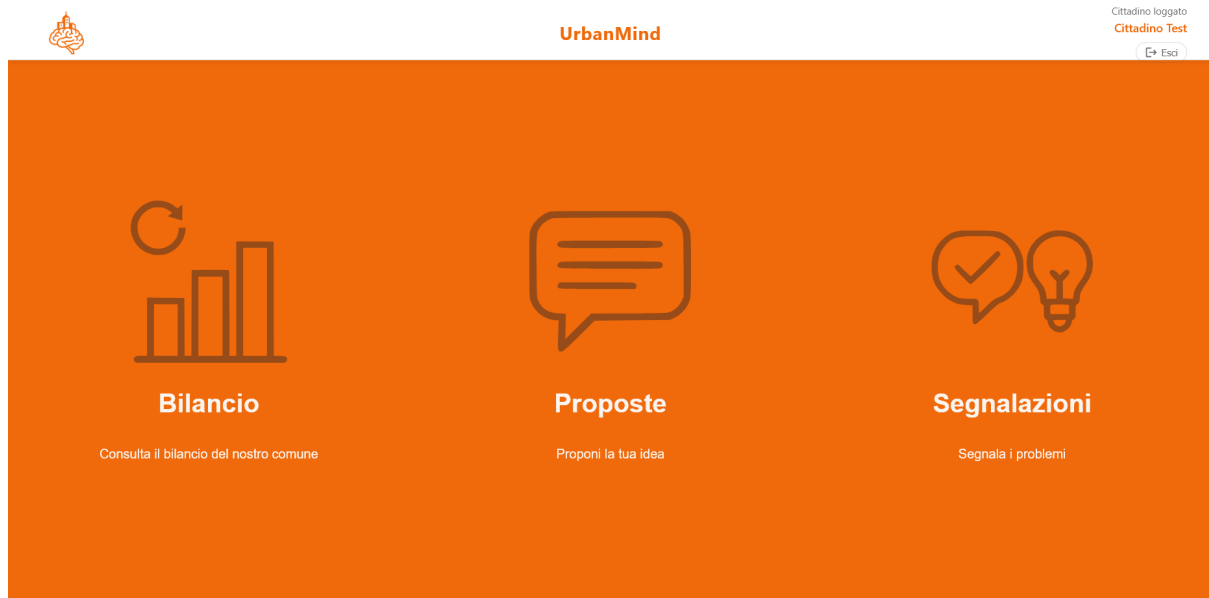


Figura 3: Dashboard Utente (Home)

La dashboard costituisce la homepage personalizzata per i cittadini. Questa pagina fornisce una panoramica generale delle attività dell'utente e del comune, presentando informazioni aggregate e collegamenti rapidi alle funzionalità principali. È progettata per offrire un colpo d'occhio immediato sullo stato delle proposte, dei budget disponibili e delle segnalazioni effettuate.



4. BudgetView - Visualizzazione Budget

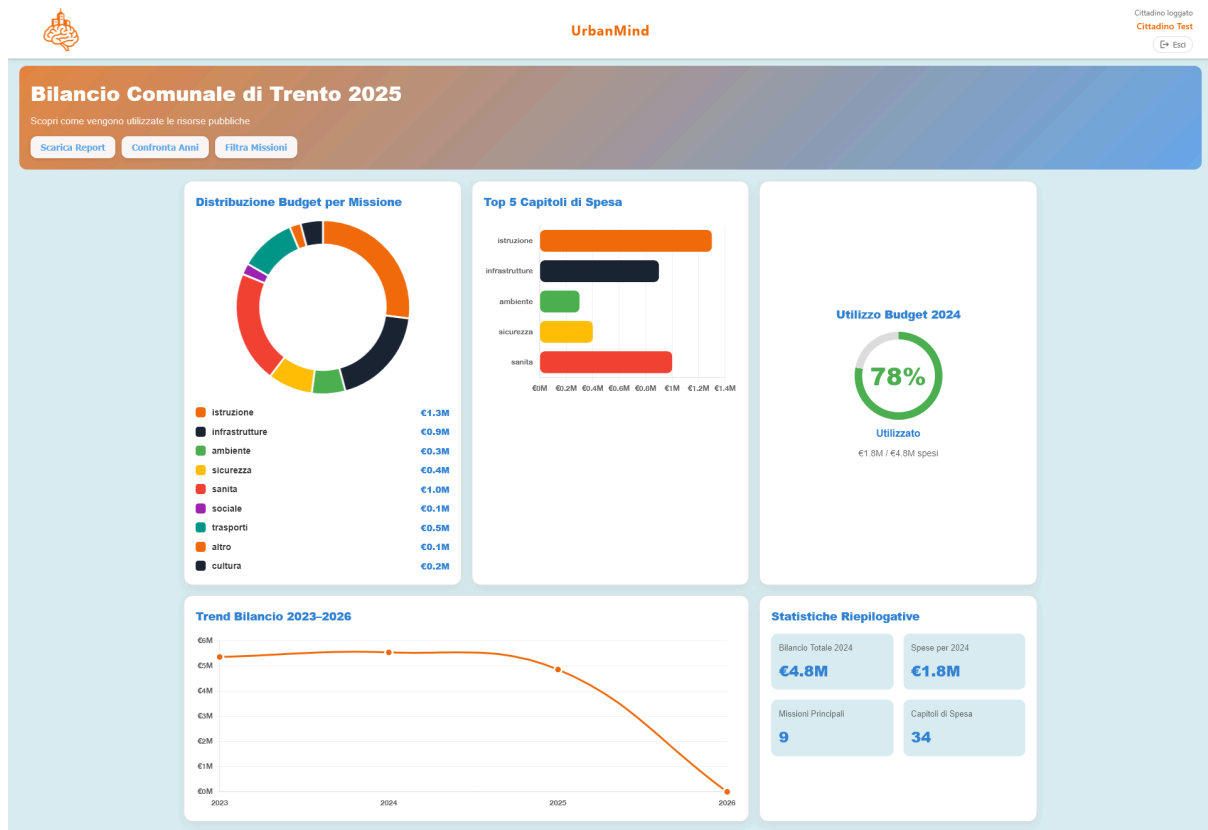


Figura 4: visualizzazione del bilancio comunale

Questa pagina permette ai cittadini di consultare i budget partecipati disponibili nel proprio comune. L'interfaccia visualizza i progetti finanziabili, le somme allocate e lo stato di avanzamento delle votazioni. I cittadini possono esplorare le diverse categorie di spesa, comprendere come vengono utilizzate le risorse pubbliche e partecipare attivamente alle decisioni di allocazione del budget comunale attraverso meccanismi di voto democratico.



5. ProposalsView - Gestione Proposte

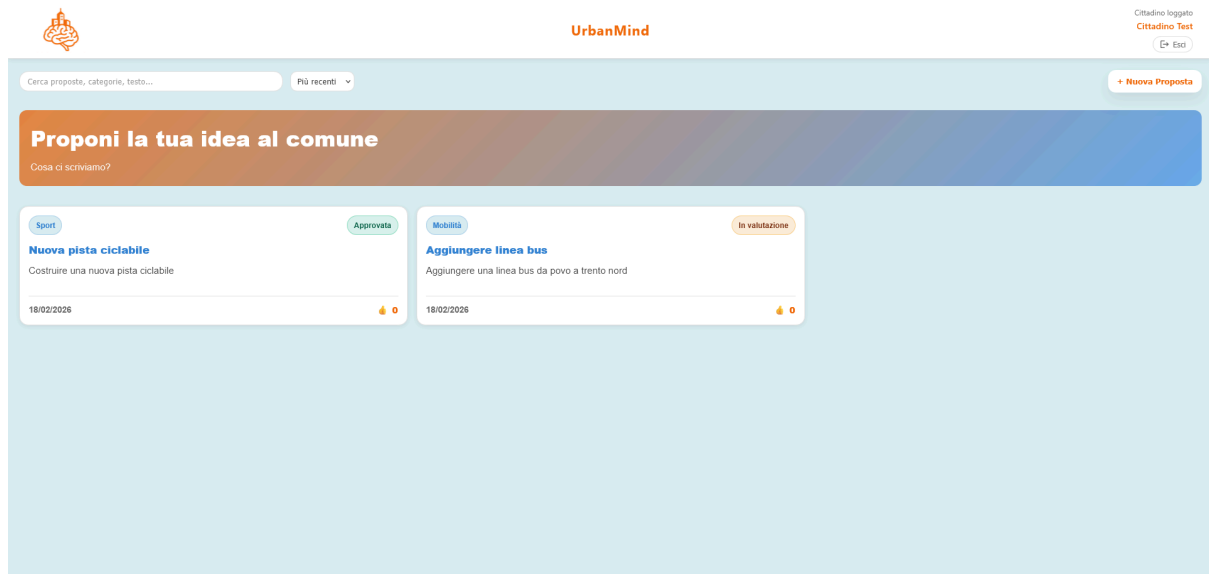


Figura 5: bacheca delle proposte (Bilancio Partecipativo)

La pagina delle proposte offre una vista completa di tutte le proposte create dall'utente o presenti nel sistema. I cittadini possono consultare le proprie proposte, verificarne lo stato (in attesa, approvate, rifiutate), votare proposte di altri cittadini e monitorare il progresso delle iniziative più popolari. L'interfaccia include funzionalità di ricerca, filtro e ordinamento per facilitare la navigazione tra molteplici proposte.

6. NewProposalView - Creazione Nuova Proposta

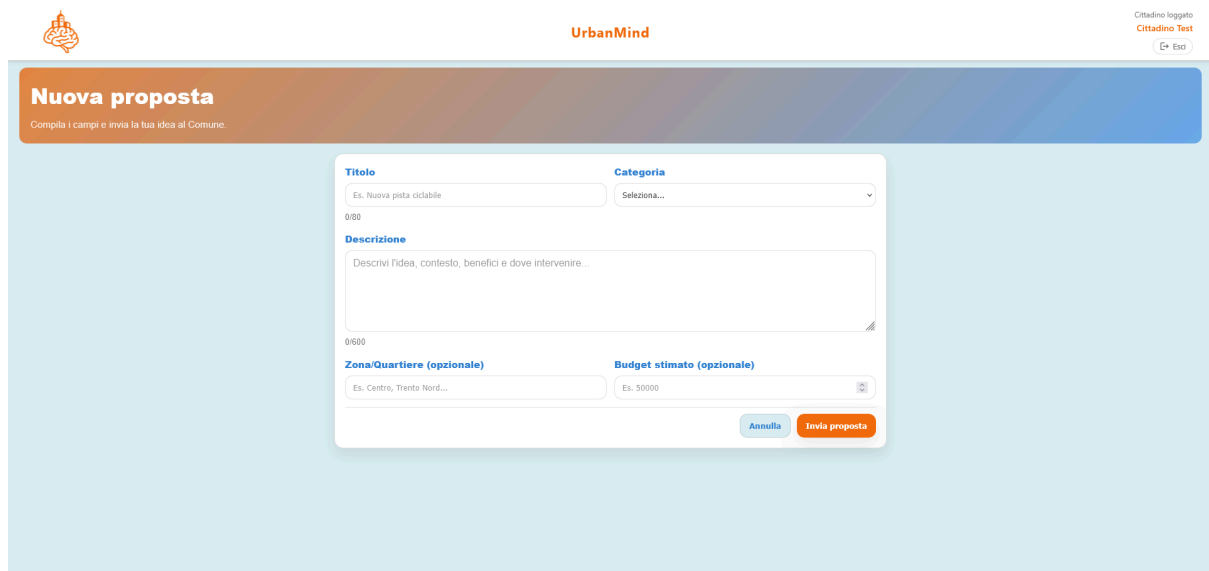


Figura 6: form di inserimento proposta

Questa pagina consente ai cittadini di sottoporre nuove proposte al comune. Il form guidato richiede informazioni dettagliate come titolo, descrizione, categoria di intervento, localizzazione geografica (probabilmente integrata con mappe interattive Leaflet) e budget stimato. L'interfaccia è progettata per essere intuitiva e accessibile,



guidando l'utente passo dopo passo nella compilazione e validando i dati inseriti prima dell'invio.

7. NewReportView - Creazione Segnalazione

The screenshot shows the 'Nuova segnalazione' (New Report) form in the UrbanMind application. The form is titled 'Nuova segnalazione' with a subtitle 'Descrivi il problema, indica la zona e (se vuoi) allega una foto.' The form fields include: 'Titolo' (Title) with a placeholder 'Es. Buche sulla strada', 'Categoria' (Category) with a dropdown menu, 'Descrizione' (Description) with a text area and placeholder 'Cosa succede? Da quanto tempo? Ci sono rischi per pedoni/auto?', 'Zona/Indirizzo' (Zone/Address) with a placeholder 'Es. Via Roma 12, Trento', 'Priorità' (Priority) with a dropdown menu, and 'Foto (opzionale)' (Optional Photo) with a file upload button. The form also includes 'Annulla' (Cancel) and 'Invia segnalazione' (Send Report) buttons. The top of the page shows the UrbanMind logo and a user login status 'Cittadino loggato Cittadino Test'.

Figura 7: form di invio segnalazione

I cittadini possono utilizzare questa pagina per inviare segnalazioni riguardanti problematiche urbane (buche stradali, illuminazione non funzionante, rifiuti abbandonati, etc.). Il sistema di segnalazione permette di allegare foto, specificare la posizione esatta tramite mappa interattiva e categorizzare il tipo di problema. Questa funzionalità rappresenta un canale diretto di comunicazione tra cittadini e amministrazione comunale.



Area Operatore

L'area operatore è dedicata al personale comunale e include cinque sezioni amministrative:

8. OperatorDashboardView - Dashboard Operatore

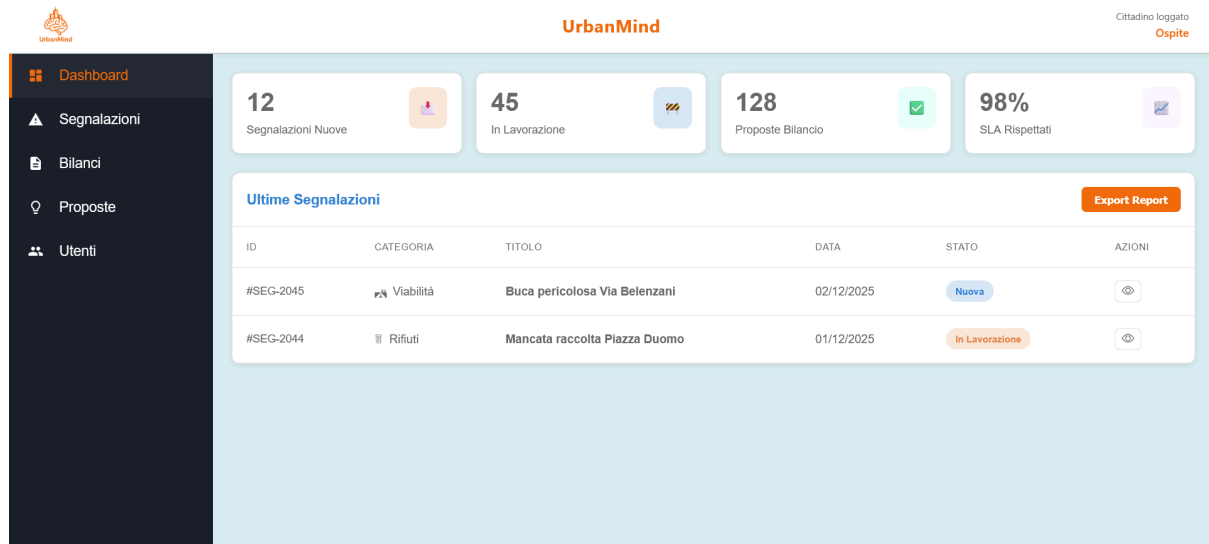


Figura 8: Dashboard Amministratore (Admin)

La dashboard amministrativa fornisce agli operatori comunali una visione d'insieme completa delle attività del sistema. Include statistiche aggregate, grafici (implementati con Chart.js e vue-chartjs) che visualizzano trend, metriche di partecipazione cittadina, e alert su elementi che richiedono attenzione immediata.



9. OperatorUsersView - Gestione Utenti

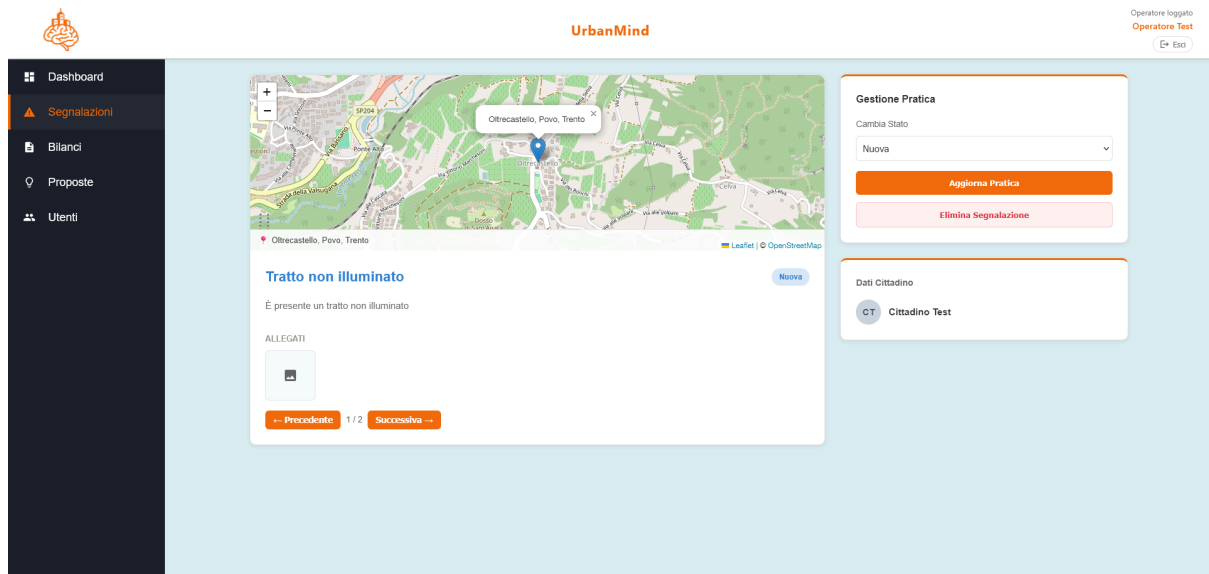


Figura 9: gestione dettaglio segnalazione (Admin)

Questa pagina amministrativa permette agli operatori di visualizzare l'elenco completo degli utenti registrati, gestire i loro profili, modificare permessi e ruoli, e monitorare l'attività degli utenti sulla piattaforma. Include funzionalità di ricerca avanzata, filtri per ruolo e stato, ed esportazione dati per analisi.

10. OperatorReportsView - Gestione Segnalazioni

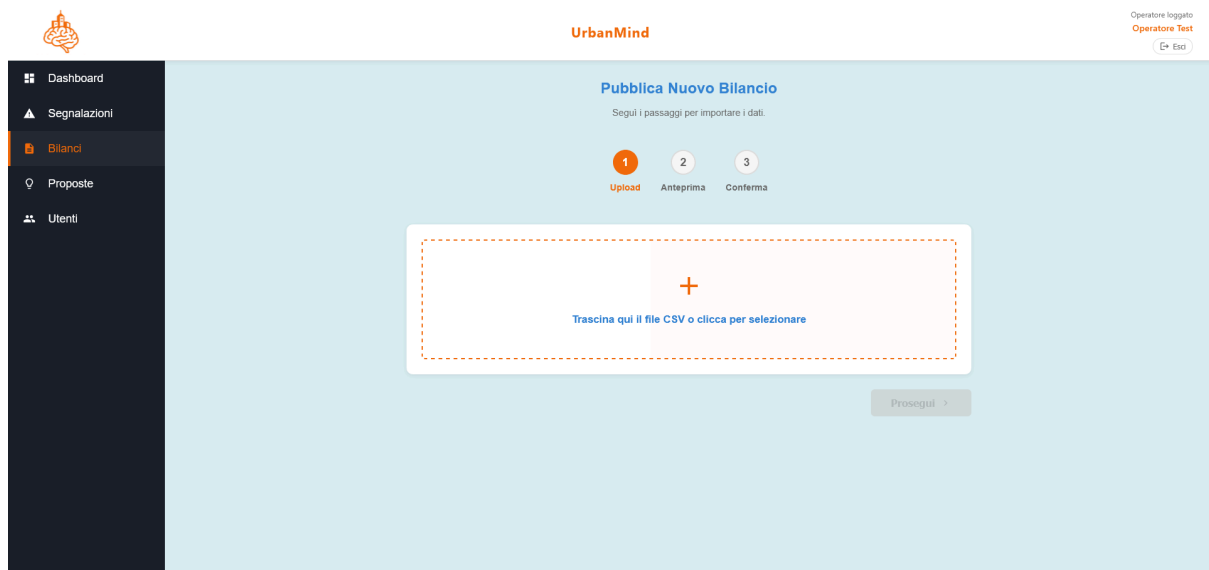


Figura 10: interfaccia di pubblicazione bilancio (Admin)

Gli operatori utilizzano questa interfaccia per processare le segnalazioni inviate dai cittadini. La pagina permette di visualizzare tutte le segnalazioni, filtrarle per stato (nuove, in lavorazione, risolte), assegnarle a specifici uffici o tecnici, aggiornarne lo stato e comunicare con i cittadini riguardo alla risoluzione dei problemi segnalati.



11. OperatorBudgetView - Gestione Budget

UrbanMind

Operatore loggato
Operatore Test

Dashboard
Segnalazioni
Bilanci
Proposte
Utenti

Proposte

Filtra per: Tutti gli stati Tutte le categorie

SPORT Nuova pista ciclabile Iniziativa il 18/02/2026

Costruire una nuova pista ciclabile

Cittadino Test 0 Support

Approvata Approvata

Dettagli Elimina

MOBILITÀ Aggiungere linea bus Iniziativa il 18/02/2026

Aggiungere una linea bus da povo a Trento nord

Cittadino Test 0 Support

In Valutazione In Valutazione

Approva Rifiuta Dettagli Elimina

Figura 11: pannello di moderazione proposte (Admin)

Questa sezione amministrativa consente agli operatori di creare nuovi budget partecipativi, caricare documenti di bilancio (supporta file CSV), definire categorie di spesa, stabilire tempistiche per le votazioni e monitorare l'andamento delle allocazioni. Include strumenti di visualizzazione grafica per analizzare la distribuzione dei fondi e le preferenze dei cittadini.

12. OperatorProposalsView - Gestione Proposte

UrbanMind

Operatore loggato
Operatore Test

Dashboard
Segnalazioni
Bilanci
Proposte
Utenti

Lista Utenti Registrati

Cerca utente... + Nuovo

UTENTE	EMAIL	RUOLO	STATO	REGISTRATO IL	AZIONI
AT Admin Test	admin@urbanmind.it	Cittadino	Attivo	18/02/2026	[Icona] + -
CT Cittadino Test	cittadino@urbanmind.it	Cittadino	Attivo	18/02/2026	[Icona] + -
OT Operatore Test	operatore@urbanmind.it	Operatore	Attivo	18/02/2026	[Icona] + -

Indietro 1 Avanti

Figura 12: gestione utenti (Admin)

La pagina di gestione proposte permette agli operatori di revisionare tutte le proposte cittadine, valutarne la fattibilità tecnica ed economica, approvarle o respingerle con motivazioni dettagliate, e monitorare l'implementazione delle



proposte approvate. L'interfaccia offre strumenti di valutazione, possibilità di richiedere integrazioni ai proponenti e workflow di approvazione multi-livello.

Tecnologie e Caratteristiche Tecniche

Il frontend implementa diverse tecnologie e best practices:

- **Leaflet**: per le mappe interattive e la geolocalizzazione di proposte e segnalazioni
- **Chart.js e vue-chartjs**: per la visualizzazione di dati statistici e grafici
- **Pinia**: gestione dello stato centralizzata per dati utente, autenticazione e cache
- **Zod**: validazione robusta degli input utente e type-safety
- **Design Responsive**: layout fluidi con uso estensivo di CSS moderno (clamp, min, max, custom properties)
- **Accessibilità**: focus management, ARIA labels, supporto tastiera
- **Performance**: lazy loading delle route, code splitting automatico con Vite

Il sistema di design utilizza variabili CSS personalizzate per colori, spaziature, bordi e transizioni, garantendo consistenza visiva in tutta l'applicazione e facilitando eventuali personalizzazioni tematiche future.

4 Deployment

Questa sezione descrive la configurazione dell'infrastruttura di **deployment** su Render, organizzata per gestire separatamente il backend e il frontend del progetto. Configurazione Infrastruttura (render.yaml)

Il file definisce l'automazione del rilascio online attraverso due servizi web distinti:

- **Backend (urbanmind-backend)**:
 - Configura un ambiente **Node.js** in modalità *production* all'interno della cartella **backend**.
 - Gestisce in sicurezza le variabili d'ambiente critiche, inclusa la connessione al database tramite **MONGODB_URI** e la generazione automatica della chiave **JWT_SECRET** per l'autenticazione.
- **Frontend (urbanmind-frontend)**:
 - Esegue il processo di **build** dell'applicazione Vue.js e serve i file statici.
 - Collega l'interfaccia alle API remote impostando la variabile **VITE_API_URL** sull'endpoint del backend pubblicato.
- **Gestione Monorepo**: Entrambi i servizi utilizzano la proprietà `rootDir` per isolare correttamente le dipendenze e i comandi di avvio delle due componenti.

Questa architettura permette di passare dallo sviluppo locale alla pubblicazione in cloud in modo trasparente e scalabile.